PWB – Microprogram Controller

Andreas Skånning (s241123), Jonas Beck Jensen (s240324)
Mads Rudolph (s246132), Sigurd Hestbech Christiansen (s245534)

April 2026

Kursus 62711 – Design of Digital Systems

Andreas
s241123

Mads
s246132

Jonas
s240324

Sigurd
s245534

Gruppe 3

DTU Elektro
Diplomingeniør, Elektro

Contents

1 Introduktion **3**

2 Design Overblik **3**

3 Program Counter **4**
3.1 Design . . . 4
3.2 VHDL Implementering . . . 6
3.3 Test . . . 7

4 Instruction Register **7**
4.1 Design . . . 7
4.2 VHDL Implementering . . . 9
4.3 Test . . . 10
4.4 Sign Extender . . . 10
4.4.1 Design . . . 10
4.4.2 Implementering . . . 11
4.4.3 Test . . . 11
4.5 Zero Filler . . . 11
4.5.1 Design . . . 11
4.5.2 Implementering . . . 12
4.5.3 Test . . . 12

5 Instruction Decoder / Controller **12**
5.1 Arkitektur . . . 12
5.2 State Machine Design . . . 12
5.3 Instruktionsoversigt . . . 13
5.4 Control State Design . . . 14
5.5 VHDL Implementering . . . 14
5.6 Test . . . 15
5.6.1 Testmetode . . . 15

6 Microprogram Controller (Top-Level) **15**
6.1 Intern Signalflow . . . 15
6.2 VHDL Implementering . . . 16
6.3 Test . . . 17
6.3.1 Testmetode . . . 17
6.3.2 Testgrupper . . . 17

7 Syntese og Simulering **17**
7.1 IDC Simulering . . . 17
7.1.1 Gruppe 1: Register-Register Instruktioner . . . 17
7.1.2 Gruppe 2: Memory & Immediate . . . 18
7.1.3 Gruppe 3: Branch & Jump . . . 18
7.1.4 Gruppe 4: Multi-cycle (LRI, SRM, SLM) . . . 19
7.1.5 Verifikationsresultat . . . 19
7.2 MPC Top-Level Simulering . . . 19
7.2.1 Gruppe 1: Register-Register Instruktioner . . . 20
7.2.2 Gruppe 2: Memory & Immediate . . . 20
7.2.3 Gruppe 3: Branch & Jump . . . 20
7.2.4 Gruppe 4: Multi-cycle . . . 21
7.2.5 Verifikationsresultat . . . 21
7.3 Verificering af Timing Diagram . . . 22

8 Konklusion **22**

1 Introduktion

2 Design Overblik

PWB er en mikroprogram-controller, som styrer hvilke mikrooperationer der skal udføres ved at udskrive et kontrolord. Den består af 3 moduler som er Program counter, Instruction register og Instruction decoder/controller.

Program Counteren (PC) er et register, der indeholder adressen på den næste instruktion, som skal hentes fra hukommelsen. PC kan enten fastholde sin værdi, inkrementeres sekventielt, opdateres relativt ved en branch eller indlæses direkte ved et jump. Funktionen styres af et 2-bit kontrolsignal, PS, som bestemmer PC'ens opførsel i hver clockcyklus.

Instruction registeret (IR) holder en 16 bit instruktion under en eksekveringscyklus. Den henter en instruktion ved at læse fra hukommelsen når Load (IL) er logisk høj.

Instruction decoder/controller (IDC) består af et sekventielt kontrolkredsløb, som forsyner et kontrolord, der bruges til mikro-operationer, som fortolker instruktionerne fra IR. En instruktion består af 16 bit opdelt i fire felter, hvor et 7-bit opcode angiver operationen, og de resterende felter refererer til registre, operander eller adresser i datapathen.

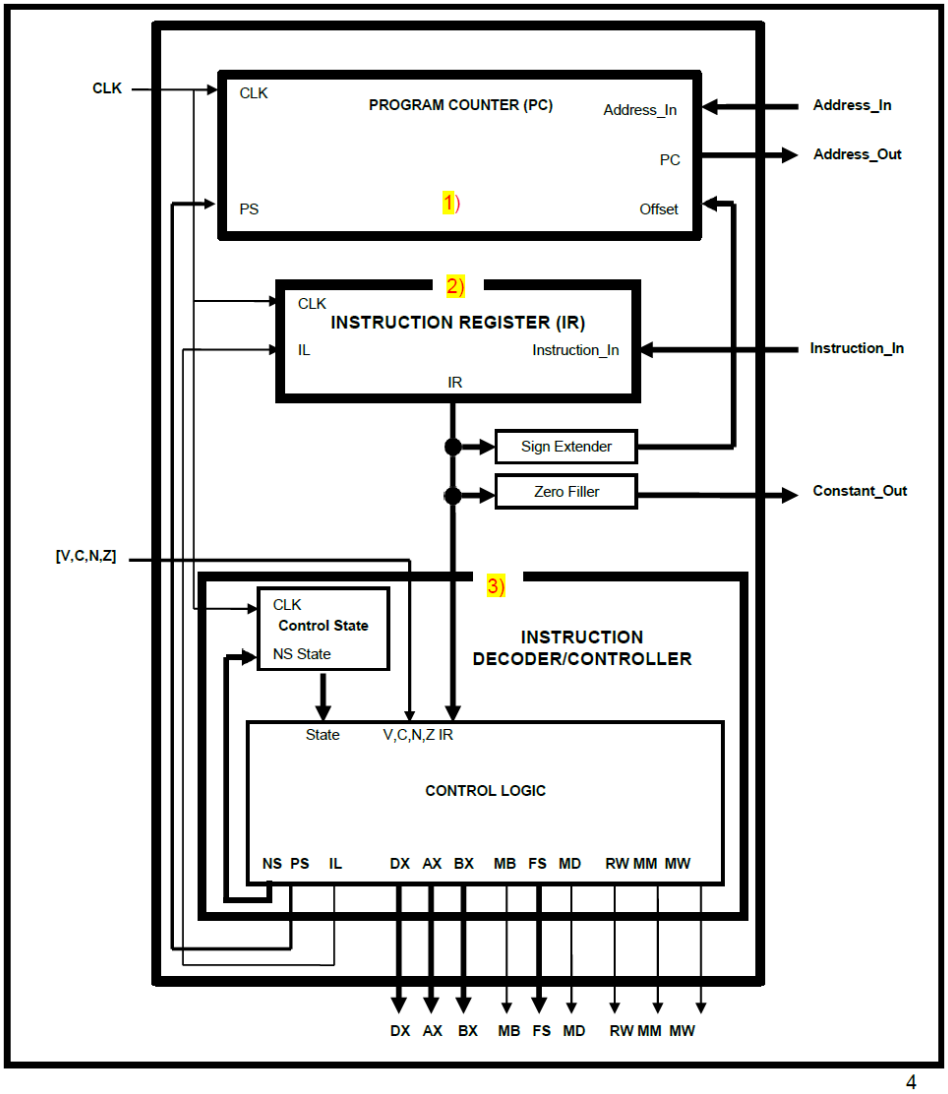


Figure 1: Blokdiagram for PWB topmodul

3 Program Counter

3.1 Design

Program Counteren (PC) er designet med udgangspunkt i figur 6-14 fra *Logic and Computer Design Fundamentals* af M. Morris Mano og Charles R. Kime. Designet opfylder desuden funktionskravene specificeret i projektbeskrivelsen for Project Work B, herunder de fire operationstilstande styret af Program Select-signalet (PS).
PC-modulet består af et 8-bit register, en adder samt en multiplexer, som tilsammen muliggør hold, inkrementering, branch og jump. Valget af næste PC-værdi foretages af multiplexeren baseret på PS-signalet, hvorefter værdien indlæses i PC-registeret på en rising edge af CLK-signalet.
Figur 2 viser det overordnede blokdiagram for Program Counteren, dog er implementeringen udviddet til 8 bit, fremfor de 4 bit der ses på figuren. Figur 3 viser den tilhørende sandhedstabel.
Når PS = 00, holdes PC-registerets nuværende værdi. Når PS = 01, inkrementeres PC med én ved hver opadgående flanke af CLK, (PC $\leftarrow$ PC + 1). Når PS = 10, udføres en branch-operation, hvor PC opdateres ved at addere et 8-bit offset til den nuværende PC-værdi (PC $\leftarrow$ PC + Offset). Offsetet kommer fra signextenderen.
Når PS = 11, udføres et jump, hvor PC-registeret indlæses med en ekstern adresse *Address_In*.
PC-modulet er desuden forsynet med et RESET-signal, som ved aktivering nulstiller PC-registeret til 0 uafhængigt af PS-signalet.
Samlet set følger Program Counterens funktionalitet direkte sandhedstabellen vist i figur 3 og opfylder kravene til styring af instruktionssekvensen.

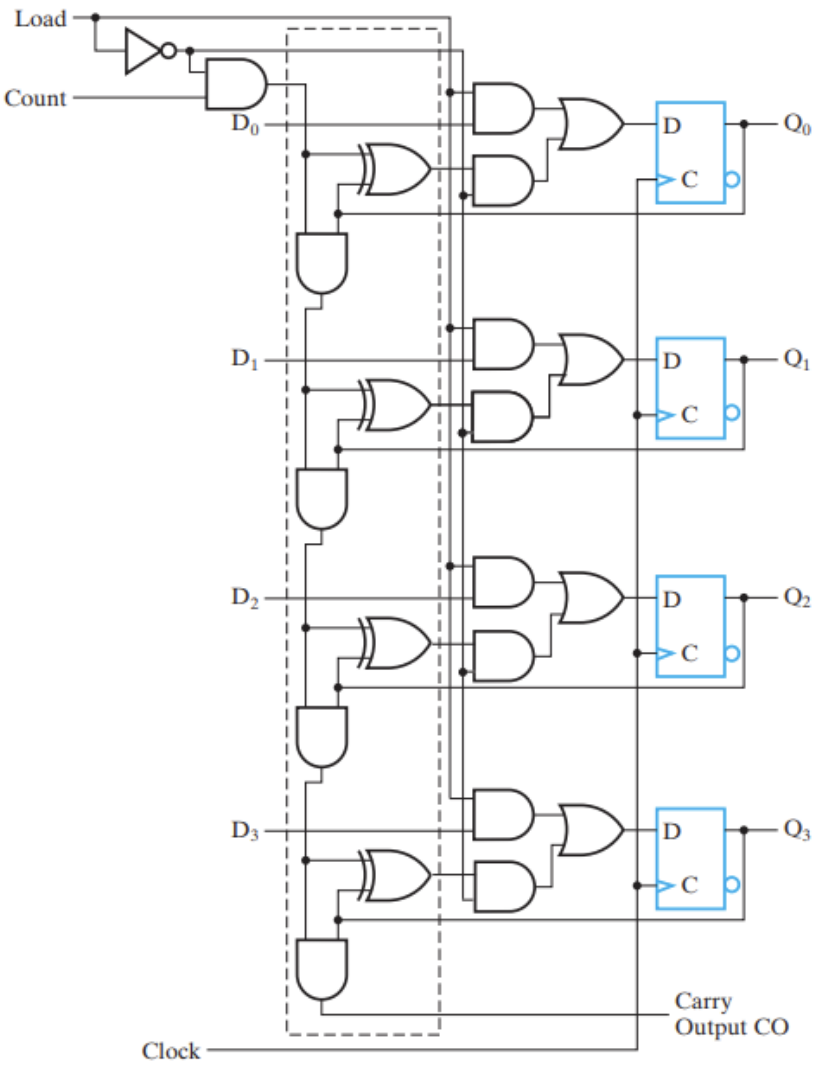


FIGURE 6-14
4-Bit Binary Counter with Parallel Load

Figure 2: Program Counter funktionsdiagram, Figure 6-14, Logic and Computer Design Fundamentals Fifth Edition, M. Morris Mano, Charles R. Kime, Tom Martin

PS	CLK	Action	Value of PC (=Address_Out)
00	↑	Hold PC	PC
01	↑	Increment PC	PC←PC+1
10	↑	Branch	PC←PC+Offset
11	↑	Jump	PC←Address_In
XX	1 or 0	Idle	PC

Figure 3: Sandhedstabel for PC, tabellen er opgivet i opgavebeskrivelsen

nedenfor ses et timing diagram for PC (figur 4), som viser hvordan PC forventes at opfører sig.

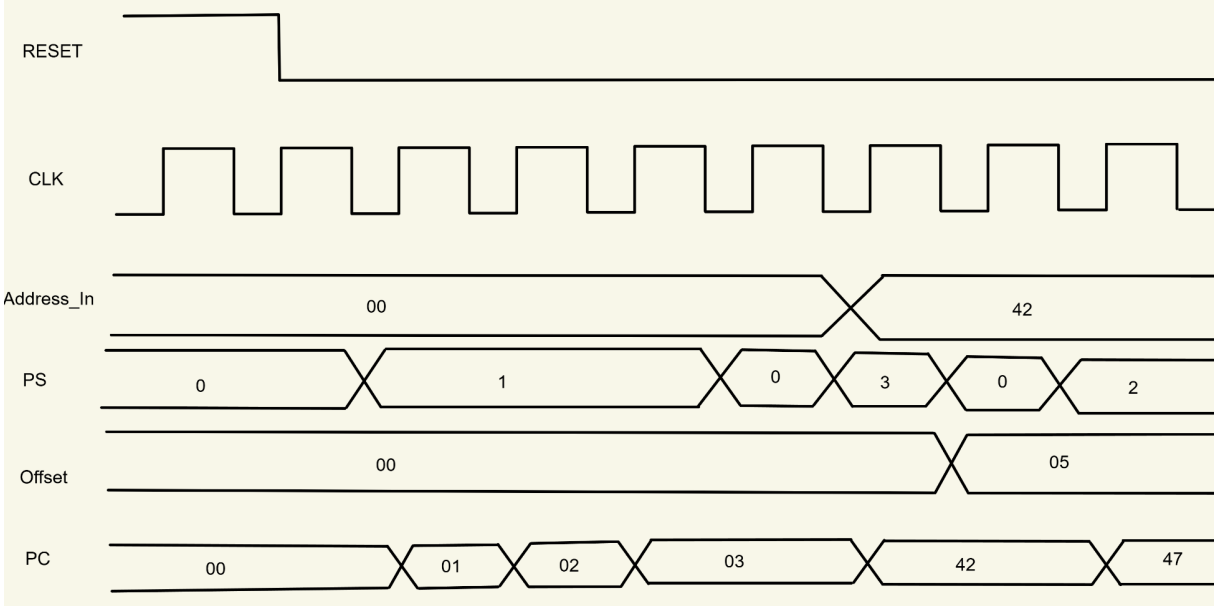


Figure 4: Timing diagram for PC

3.2 VHDL Implementering

Program Counteren er implementeret strukturelt i VHDL og består af otte identiske 1-bit CounterLogic-moduler, som tilsammen danner et 8-bit register med tælle- og load-funktionalitet. Denne opbygning følger funktionsdiagrammet for et synkront binært tælleregister, vist i figur 2.
Hvert CounterLogic-modul indeholder en D-flipflop samt kombinatorisk logik, der muliggør tre operationer: hold, inkrementering og parallel load. Modulerne er koblet i en ripple-carry struktur, hvor carry-out-signalet (CO) fra hvert bit føres videre til carry-in (Cin) på det næste bit. Dette gør det muligt at inkrementere PC-værdien med 1 ved brug af carry-kæden.
PC-registerets aktuelle værdi gemmes i signalet `PCsig`, som samtidigt anvendes som operanden i en ekstern 8-bit fulladder. Fulladderen bruges udelukkende til branch-operationen, hvor offsetværdien lægges til den nuværende PC-værdi (PC ← PC + Offset).
Signalerne `Load` og `Count` genereres udelukkende ud fra Program Select-signalet (PS). Når PS er lig 10 eller 11, aktiveres `Load`, således at PC-registeret indlæses parallelt. Når PS er 01, aktiveres `Count`, hvilket medfører inkrementering af PC med én via carry-kæden. Ved PS = 00 er både Load og Count inaktive, og PC-registeret holder sin nuværende værdi.
En 8-bit multiplexer (`MUXP`) anvendes til at vælge den værdi, der indlæses i PC-registeret ved load-operationer. Ved PS = 10 vælges summen fra fuldadderen (PC + Offset), svarende til en branch-instruktion. Ved PS = 11 vælges den eksterne adresse `Address_In`, hvilket realiserer en jump-instruktion. Ved øvrige PS-kombinationer er multiplexerens output uden betydning, da load ikke er aktiv.
Reset-signalet er ført til samtlige CounterLogic-moduler og nulstiller PC-registeret ved aktivering. Det samlede output `PC` drives direkte af `PCsig`.
Denne strukturelle implementering sikrer en direkte og gennemsigtig realisering af PC'ens funktionalitet i overensstemmelse med den specificerede sandhedstabel og projektkravene.

3.3 Test

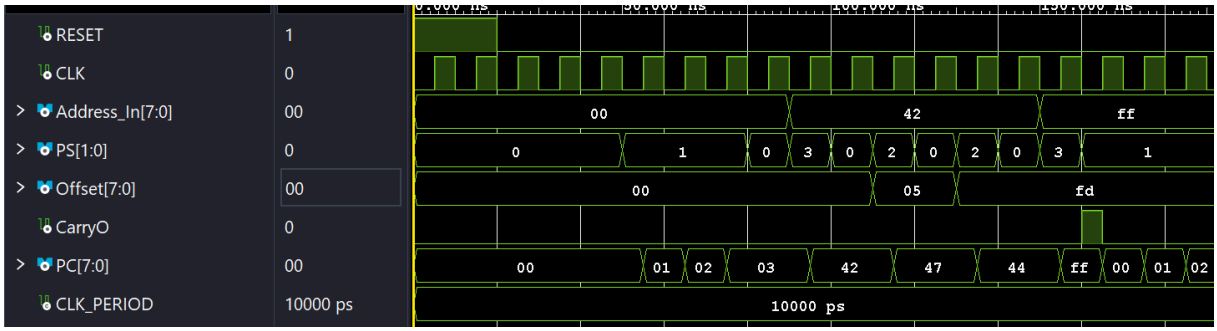


Figure 5: Testbench for PC

Testbenchen viser, at PC virker som forventet.

4 Instruction Register

4.1 Design

Instruction Register, er et register som holder en 16 bit instruktion fra hukommelsen. Den styres af et load signal (IL), hvis dette signal er logisk højt læses den 16 bit instruktion ind fra hukommelsen og hvis den er logisk lav, så holdes den sidst indlæste instruktion. Sandhedstabellen for den ønskede funktionalitet ses nedenfor og er taget fra opgavebeskrivelsen:

IL	CLK	Action	Value of IR
0	↑	No Load	IR
1	↑	Load Instruction	IR←Instruction_In
X	1 or 0	Idle	IR

Figure 6: Sandhedstabel, Instruction Register

For at designe et modul som kan udføre dette bruges et 16-bit register lavet af d-flip flops. For at styre registerets indlæsning, bruges som nævnt i forrige afsnit et load (IL) som styrer om der bliver loadet en instruktion ind ved en høj clock kant eller ikke, dette sikrer at registeret kun indlæser i registeret når det IL er logisk høj. Instruction_In ledes direkte til D på D-flip floppen som så ved næste opadgående flanke på clk, hvor IL er høj kommer ud på udgangen af Q.

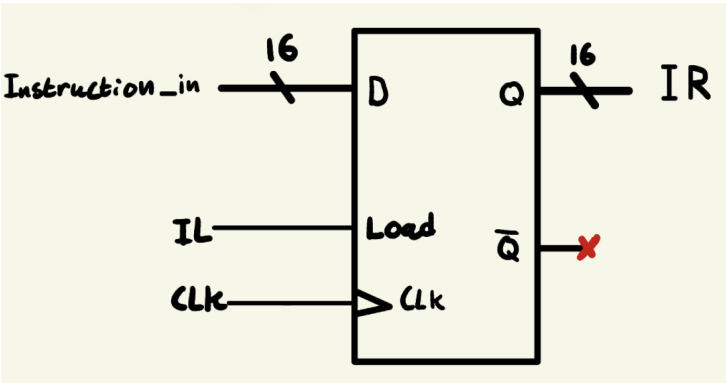


Figure 7: Instruction register moduldiagram

Herunder ses et timing-diagrammer for modulet, der gennemløber alle de mulige funktioner som vi vil have modulet til at kunne udføre. Disse timing diagrammer sammenholdes med testbenches som ligger under test, og hvis modulet virker som det skal vil disse være ens.
Disse diagrammer skal ses som et enkelt timing diagram som er delt op i 3 diagrammer.

I denne del som man kan se på Timing diagrammet skal vi kunne sikre at systemet ikke loader instruction register (IR) med Instruction_in når Load (IL) er logisk lav.

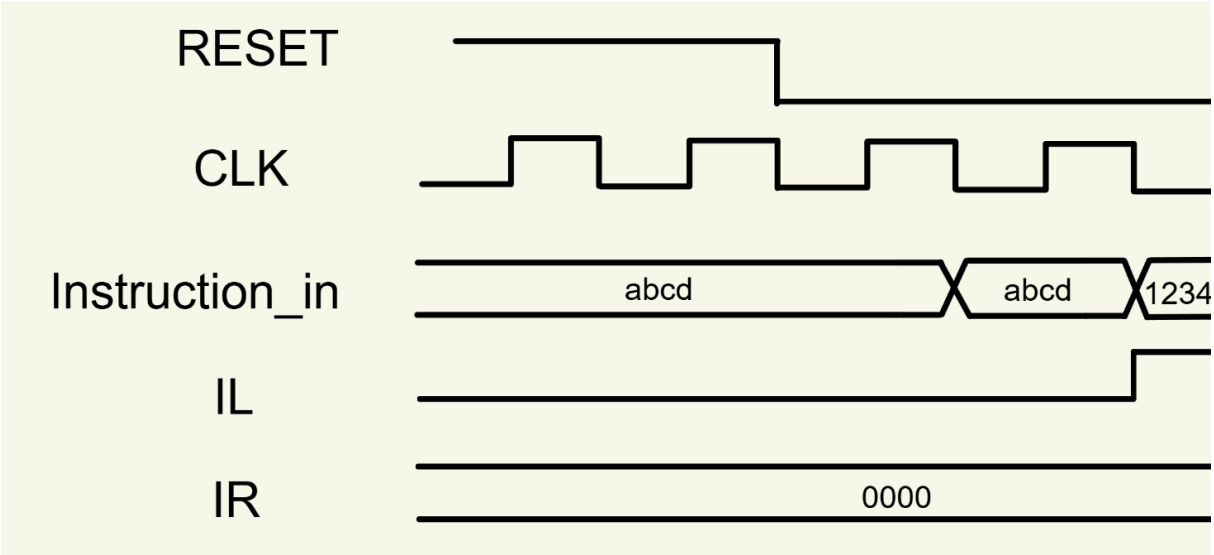


Figure 8: Timing IR del 1

I denne del af timing diagrammet skal vi kunne sikre at systemet kun loader instruction register (IR) når IL er logisk høj og at det kun sker på en opadgående CLK-flanke, samt at det asynkrone RESET kan nulstille registeret lige så snart RESET signalet går højt uagtet om der er en opadgående CLK-flanke.

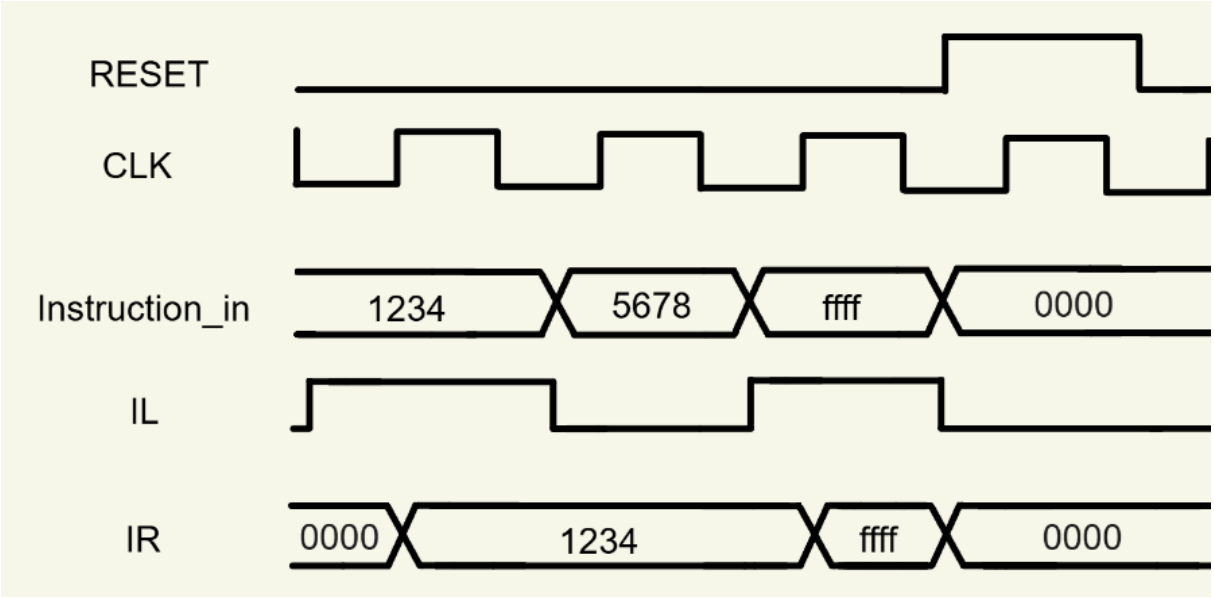


Figure 9: Timing IR del 2

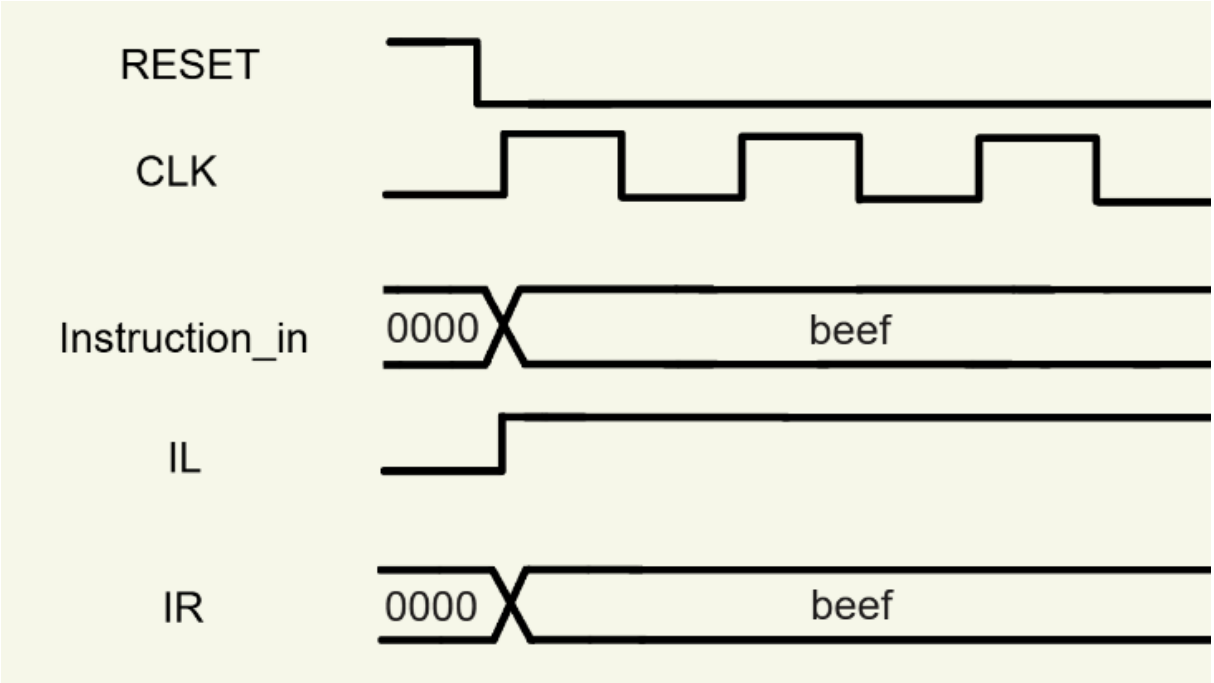


Figure 10: Timing IR del 3

4.2 VHDL Implementering

Instruction Registeret (IR) er implementeret som et 16-bit synkront register med load-enable og reset-funktionalitet. Implementeringen er baseret på et tidligere anvendt D-flip-flop-design, som er udvidet fra ét bit til 16 bit. Dette muliggør lagring af hele instruktionen i ét samlet modul fremfor 16 separate registre.

Det underliggende modul `flip_flop_16` er modelleret behavioralt ved hjælp af en clocked process. Registeret nulstilles, når reset-signalet er aktivt, hvilket sætter alle bits i IR til nul. Ved den positive flanke af clk-signalet opdateres registerets indhold med signalet `Instruction_In`, såfremt load-signalet `IL` er høj. Hvis `IL` er lav, fastholder registeret sin nuværende værdi.

Load-signalet `IL` fungerer dermed som et enable signal og sikrer, at instruktionen kun indlæses i overensstemmelse med sandhedstabellen vist i figur 6. Når `IL = 0`, vil IR forblive uændret, også selvom der forekommer en opadgående flanke på clk.

Instruction Registeret er instansieret strukturelt i topmodulet via det generiske flip-flop-modul, hvor `Instruction_In` tilsluttes D-inputtet, `IL` til load-signalet og outputtet `IR` leverer den lagrede instruktion til Instruction Decoder/Controller og de tilhørende hjælpeblokke som Sign-Extender og Zero-Filler.

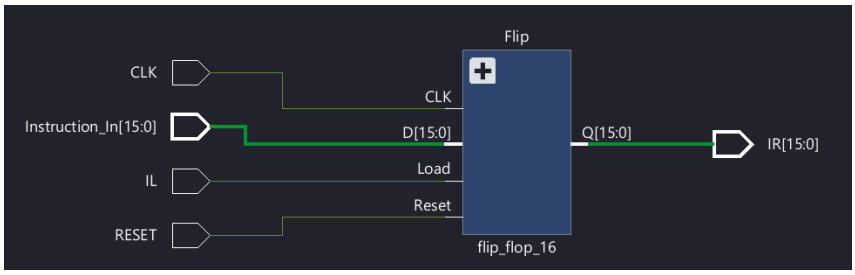


Figure 11: Instruction register VHDL-implementering

4.3 Test

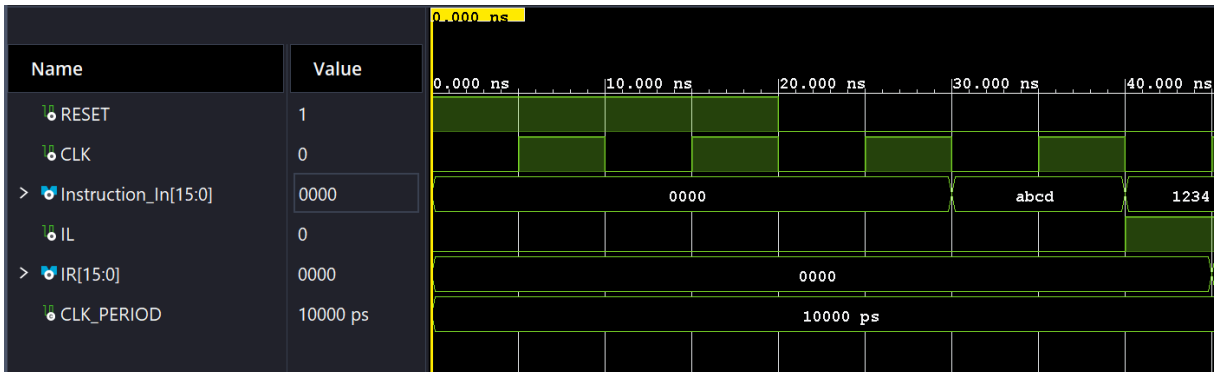


Figure 12: Testbench for IR 1

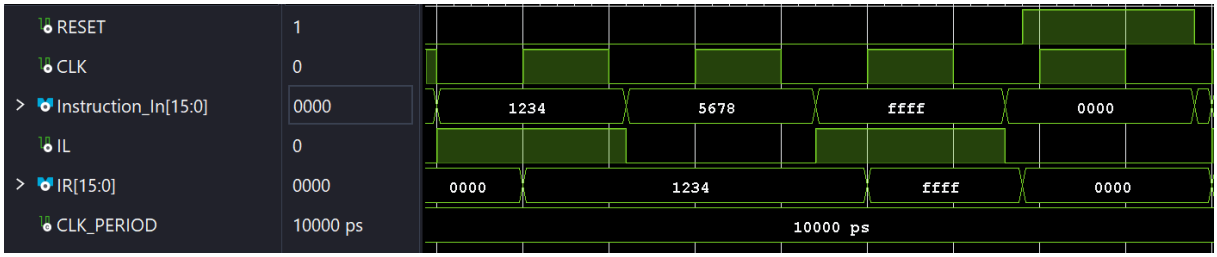


Figure 13: Testbench for IR 2

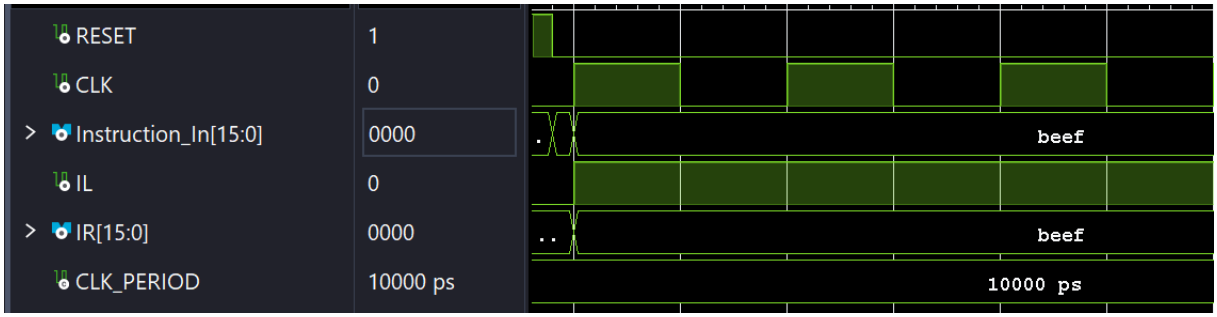


Figure 14: Testbench for IR 3

Vi kan se, at testbenchene matcher timingdiagrammerne i design afsnittet og derfor kan vi konkludere, at systemet virker som det skal.

4.4 Sign Extender

4.4.1 Design

Sign-Extenderen er designet i overensstemmelse med projektbeskrivelsen og anvendes til at generere et 8-bit offset med bevaret fortegn. Modulet benytter bit IR(8) som fortegnsbit, som kopieres til de tre mest betydende bits i outputtet. De resterende bits i outputtet dannes ud fra IR(7 downto 6) og IR(2 downto 0). Samlet kommer outputtet til at have følgende form:

$$\text{SignExtended_8} = IR(8)\, IR(8)\, IR(8)\, IR(7:6)\, IR(2:0)$$

Derved opnås en korrekt to-komplements repræsentation af instruktionens offset, som kan anvendes i branch-instruktioner.

4.4.2 Implementering

Sign-Extenderen er et rent kombinatorisk modul, som er designet til at generere en 8-bit værdi ud fra udvalgte bits i Instruction Registeret (IR) under bevarelse af to-komplements repræsentation. Modulet anvendes primært i forbindelse med branch-instruktioner, hvor et relativt offset kan være både positivt og negativt.
Sign-Extenderen benytter bit IR(8) som fortegnsbit. Denne bit kopieres til de tre mest betydende bits i outputtet, mens de resterende bits dannes ud fra IR(7 downto 6) og IR(2 downto 0). Derved opnås en korrekt fortegnsudvidelse fra instruktionens felt til et 8-bit offset.

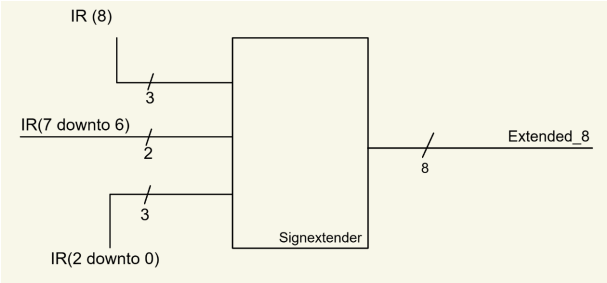


Figure 15: SignExtender funktionsdiagram

4.4.3 Test

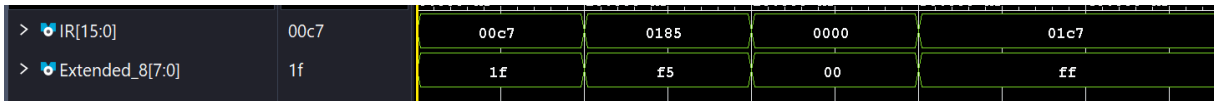


Figure 16: Testbench for SignExtender

Testbenchen viser, at modulet virker som forventet.

4.5 Zero Filler

4.5.1 Design

Zero-Filleren er et rent kombinatorisk modul, som er designet til at generere en 8-bit værdi ved nuludvidelse af en del af Instruction Registeret. Modulet benytter de tre mindst betydende bits fra IR og indsætter fem førende nuller.
Outputtet får dermed formen:

$$\text{ZeroFilled_8} = 00000\, IR(2:0)$$

Zero-Filleren Giver output fra PWB-modulet i form af **Constant_Out**

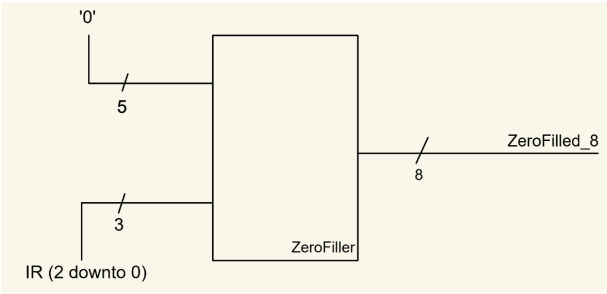


Figure 17: Zero Filler funktionsdiagram

4.5.2 Implementering

Zero-Filleren er implementeret som et rent kombinatorisk modul i VHDL. Outputsignalet `ZeroFilled_8` dannes ved at sammensætte fem konstante nuller med de tre mindst betydende bits fra inputtet `IR`. Implementeringen kræver ingen processer eller CLK-signal og realiseres som en samtidig signaltilordning.

4.5.3 Test

Figure 18: Testbench for ZeroFiller

Testbenchen viser, at modulet virker som forventet.

5 Instruction Decoder / Controller

Instruction Decoder / Controller (IDC) er det centrale styringsmodul i mikroprogramcontrolleren. Den modtager den aktuelle instruktion fra IR og statusflag (V, C, N, Z) fra datavejen, og genererer det komplette kontrolord der styrer alle andre moduler i systemet.

5.1 Arkitektur

IDC er implementeret som en Mealy-type finite state machine (FSM) med seks tilstande og en to-process model:

1. **State register** (sekventiel process): Opdaterer `current_state` på opadgående clockflanke. Asynkron reset til INF.
2. **Next-state + output logik** (kombinatorisk process): Beregner `next_state` og alle kontrolsignaler baseret på `current_state`, opcode (`IR(15:9)`) og statusflag.

Outputsignalerne der udgør kontrolordet:

Signal	Bredde	Funktion
PS	2 bit	Program Counter select (00=hold, 01=inc, 10=branch, 11=jump)
IL	1 bit	Instruction Load (load IR fra hukommelse)
DX	4 bit	Destination register adresse
AX	4 bit	A-source register adresse
BX	4 bit	B-source register adresse
MB	1 bit	MUX B select (0=register, 1=constant)
FS	4 bit	Function Unit select
MD	1 bit	MUX D select (0=ALU, 1=memory)
RW	1 bit	Register Write enable
MM	1 bit	MUX M select (0=ALU adresse, 1=PC adresse)
MW	1 bit	Memory Write enable

Table 1: IDC kontrolsignaler

5.2 State Machine Design

FSM'en har seks tilstande. De fleste instruktioner er 2-cyklus (INF → EX0 → INF), mens LRI bruger 3 cyklusser og SRM/SLM bruger en variabel shift-løkke:

Tilstand	Funktion
INF	Instruction Fetch: IL=1, MM=1, henter næste instruktion fra M[PC]
EX0	Primær eksekvering: dekoder opcode, udfører 2-cyklus instruktioner
EX1	Sekundær eksekvering: LRI anden læsning, SRM/SLM loader tæller
EX2	Shift: udfører ét shift af R8 (sr eller sl afhængig af opcode)
EX3	Tæller: R9 $\leftarrow$ R9 $-$ 1, tjekker Z-flag for løkke
EX4	Færdig: R[DR] $\leftarrow$ R8, returnerer til INF

Table 2: IDC tilstande

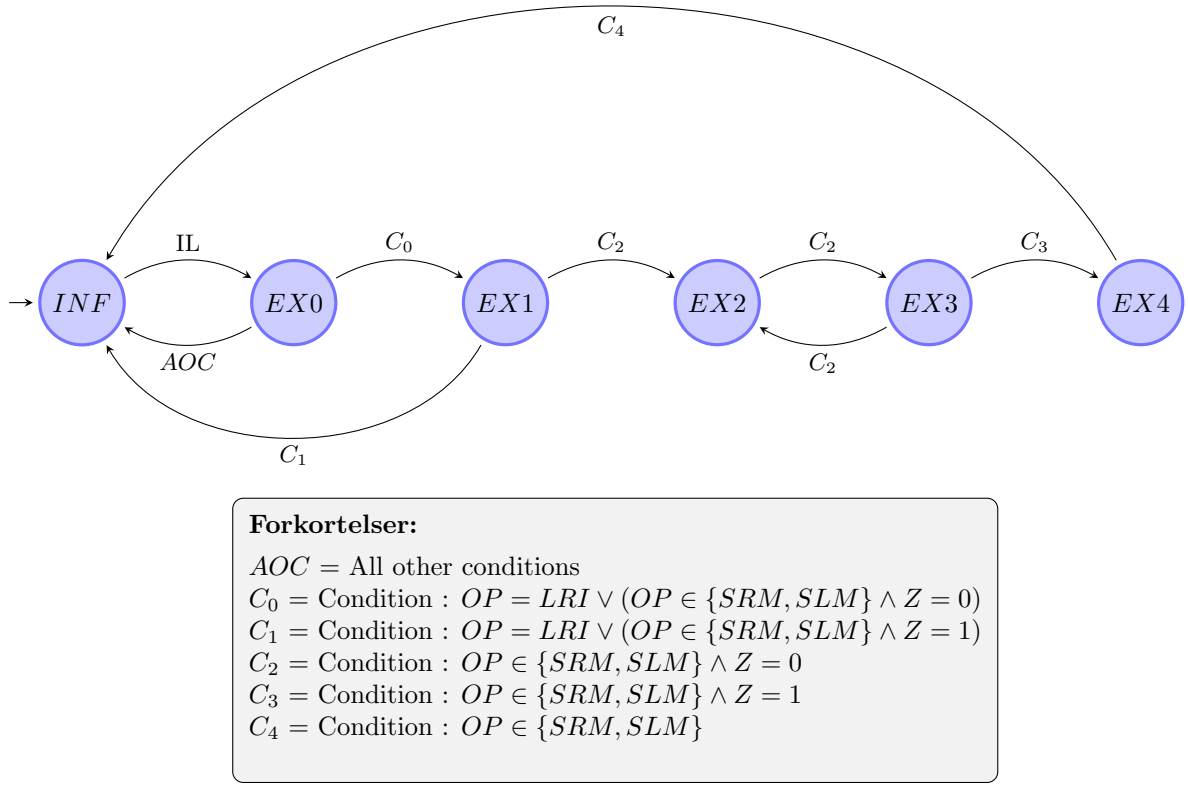


Figure 19: IDC state diagram med tilstandsovergange og betingelser.

Tilstandsovergangene for de forskellige instruktionstyper:

- **Register-register** (MOVA, ADD, SUB, etc.): INF $\rightarrow$ EX0 $\rightarrow$ INF
- **Memory** (LD, ST): INF $\rightarrow$ EX0 $\rightarrow$ INF
- **Immediate** (ADI, LDI): INF $\rightarrow$ EX0 $\rightarrow$ INF (MB=1)
- **Branch** (BRZ, BRN): INF $\rightarrow$ EX0 $\rightarrow$ INF (PS betinget af flag)
- **Jump** (JMP): INF $\rightarrow$ EX0 $\rightarrow$ INF (PS=11)
- **LRI**: INF $\rightarrow$ EX0 $\rightarrow$ EX1 $\rightarrow$ INF
- **SRM/SLM**: INF $\rightarrow$ EX0 $\rightarrow$ EX1 $\rightarrow$ EX2 $\leftrightarrow$ EX3 $\rightarrow$ EX4 $\rightarrow$ INF

5.3 Instruktionsoversigt

Tabel 3 viser kontrolordet for hver instruktion i EX0-tilstanden. For ALU-operationer gælder at FS aflæses direkte fra `IR(12:9)`.

Instruktion	Opcode	PS	FS	MB	MD	RW	MW
MOVA	0000000	01	0000	0	0	1	0
INC	0000001	01	0001	0	0	1	0
ADD	0000010	01	0010	0	0	1	0
SUB	0000101	01	0101	0	0	1	0
DEC	0000110	01	0110	0	0	1	0
OR	0001000	01	1000	0	0	1	0
AND	0001001	01	1001	0	0	1	0
XOR	0001010	01	1010	0	0	1	0
NOT	0001011	01	1011	0	0	1	0
MOVB	0001100	01	1100	0	0	1	0
LD	0010000	01	—	0	1	1	0
ST	0100000	01	—	0	0	0	1
ADI	1000010	01	0010	1	0	1	0
LDI	1001100	01	1100	1	0	1	0
BRZ	1100000	10/01	—	0	0	0	0
BRN	1100001	10/01	—	0	0	0	0
JMP	1110000	11	—	0	0	0	0
LRI	0010001	00	—	0	1	1	0
SRM	0001101	00/01	—	0	0	1	0
SLM	0001110	00/01	—	0	0	1	0

Table 3: Kontrolord for hver instruktion i EX0. Branch PS afhænger af flag.

5.4 Control State Design

Control State(CS), som udgør **state registeret**, er IDC'ens tilstandshukommelse og implementeres som et synkront register med asynkront reset. **CS**-blokken lagrer den aktuelle tilstand i FSM'en og opdaterer signalet `current_state` til værdien af `next_state` på den opadgående flanke af clock-signalet.

Ved aktivering af reset sættes `current_state` til INF tilstanden, hvilket sikrer, at systemet altid starter i Instruction Fetch fasen. **CS**-blokken indeholder ingen beslutningslogik, men fungerer udelukkende som lagring af den aktuelle tilstand, der anvendes som input til den kombinatoriske control logic.

Event	Funktion
`clk` ↑	`current_state` ← `next_state`
`reset = 1`	`current_state` ← `INF`
1 or 0	Idle

Table 4: Opdatering af Control State

5.5 VHDL Implementering

Den kombinatoriske process bruger en *case-in-case* struktur: det ydre case-statement vælger tilstand, og det indre case-statement i EX0/EX1 dekoder opcoden. Standardværdier sættes øverst i processen for at undgå latches:

```
control_logic: process(current_state, IR, V, C, N, Z)
begin
    -- Standardværdier (undgår latches)
    next_state <= INF;
    PS  <= "00";  IL  <= '0';
    DX  <= '0' & IR(8 downto 6);
    AX  <= '0' & IR(5 downto 3);
    BX  <= '0' & IR(2 downto 0);
    MB  <= '0';   FS  <= "0000";
    MD  <= '0';   RW  <= '0';
    MM  <= '0';   MW  <= '0';

    case current_state is
        when INF =>
            next_state <= EX0;
            IL <= '1';  MM <= '1';

        when EX0 =>
            case IR(15 downto 9) is
                when "0000000" | "0000001" | ... =>
```

```
                    -- ALU register-register
                    next_state <= INF;
                    PS <= "01";
                    FS <= IR(12 downto 9);
                    RW <= '1';
                ...
            end case;
        ...
    end case;
end process;
```

Listing 1: IDC kombinatorisk process (uddrag)

Et vigtigt designvalg er at FS for ALU-operationer aflæses direkte fra `IR(12:9)`. Opcoden er kodet så bits 12–9 matcher funktionskoden i ALU'en, hvilket eliminerer behovet for en separat oversættelsestabel.

5.6 Test

IDC er verificeret med en testbench opdelt i fire grupper via en `TEST_GROUP` konstant. Hver gruppe kan køres separat for at holde simuleringstiden kort og bølgeformerne overskuelige.

5.6.1 Testmetode

Testbenchen bruger VHDL `assert`-statements til automatisk verifikation. For hver instruktion sættes IR til den relevante opcode, og efter én clockcyklus (plus 1 ns delta for signalpropagering) kontrolleres de forventede kontrolsignaler. Hvis et signal afviger fra den forventede værdi, rapporterer simulatoren en fejl med instruktionsnavn og forventet værdi i TCL-konsollen.
Mønsteret for en enkelt 2-cyklus instruktion er:

```
-- Sæt IR til ADD-instruktion (opcode=0000010, DR=001, SA=010, SB=011)
IR <= make_ir("0000010");
wait until rising_edge(CLK); wait for 1 ns;  -- EX0 outputs stabile
assert FS = "0010" report "ADD: FS expected 0010" severity error;
assert RW = '1'    report "ADD: RW expected 1"    severity error;
assert PS = "01"   report "ADD: PS expected 01"   severity error;
wait until rising_edge(CLK);  -- Tilbage til INF
```

Listing 2: Testbench assert-mønster (eksempel: ADD)

Hjælpefunktionen `make_ir` sammensætter en 16-bit IR-værdi fra opcode og registeradresser, hvilket gør testcases kompakte og læsbare.
Hvis alle asserts består, vises kun `Note`-beskeder i TCL-outputtet. Hvis et assert fejler, vises en `Error`-besked med det specifikke signal og forventede værdi — dette gør det muligt hurtigt at lokalisere fejlen. Fraværet af `Error`-beskeder i outputtet bekræfter at alle kontrolsignaler er korrekte.
Simuleringsresultaterne for alle fire testgrupper er vist i sektion 7.1.

6 Microprogram Controller (Top-Level)

Microprogram Controlleren (MPC) er topmodulet der forbinder alle submoduler til et samlet styringskredsløb. Modulet modtager en instruktion fra hukommelsen og statusflag fra datavejen, og producerer det komplette kontrolord samt næste hukommelsesadresse.

6.1 Intern Signalflow

MPC'en forbinder fem submoduler via interne signaler som vist i figur 1. Signalflowet er som følger:

1. **Instruction Register** modtager `Instruction_In` (16 bit) fra hukommelsen og load-signalet `IL` fra IDC. Når IL er høj ved en opadgående clockflanke, indlæses instruktionen i IR. Det interne signal `IR_sig` (16 bit) distribueres til tre moduler: Sign Extender, Zero Filler og IDC.

2. **Sign Extender** modtager `IR_sig` og producerer et 8-bit fortegnsudvidet offset (`Extended_sig`), som føres til Program Counterens offset-input. Dette bruges ved branch-instruktioner (PS=10).

3. **Zero Filler** modtager `IR_sig` og producerer et 8-bit nulfyldt konstantfelt, som føres direkte til topmodulets `Constant_Out`. Dette bruges af datavejen ved immediate-instruktioner (ADI, LDI).

4. **Program Counter** modtager `PS_sig` (2 bit) fra IDC, `Extended_sig` fra Sign Extenderen og `Address_In` fra topmodulets input. PC-outputtet drives direkte til `Address_Out` uden mellemliggende signal.

5. **IDC** modtager `IR_sig` og statusflag (V, C, N, Z) og genererer alle kontrolsignaler. De interne signaler `PS_sig` og `IL_sig` bruges til styring af henholdsvis PC og IR. De resterende kontrolsignaler (DX, AX, BX, FS, MB, MD, RW, MM, MW) forbindes direkte til topmodulets udgange.

Internt signal	Bredde	Forbindelse
`IR_sig`	16 bit	IR → IDC, Sign Extender, Zero Filler
`PS_sig`	2 bit	IDC → Program Counter
`IL_sig`	1 bit	IDC → Instruction Register
`Extended_sig`	8 bit	Sign Extender → Program Counter (Offset)

Table 5: Interne signaler i MPC topmodulet

6.2 VHDL Implementering

Topmodulet er implementeret rent strukturelt ved direkte instansiering af de fem submoduler. Arkitekturen indeholder ingen processer eller kombinatorisk logik udover port-mapping. De fire interne signaler fra tabel 5 forbinder submodulerne.

```vhdl
architecture MCU_Structural of MicroprogramController is
    signal IR_sig       : std_logic_vector(15 downto 0);
    signal PS_sig       : std_logic_vector(1 downto 0);
    signal IL_sig       : std_logic;
    signal Extended_sig : std_logic_vector(7 downto 0);
begin
    PC_inst: entity work.ProgramCounter
    port map(
        RESET => RESET, CLK => CLK,
        Address_In => Address_In,
        PS => PS_sig, Offset => Extended_sig,
        Carry0 => open, PC => Address_Out
    );

    IR_inst: entity work.InstructionRegister
    port map(
        RESET => RESET, CLK => CLK,
        Instruction_In => Instruction_In,
        IL => IL_sig, IR => IR_sig
    );

    SE_inst: entity work.SignExtender
    port map(IR => IR_sig, Extended_8 => Extended_sig);

    ZF_inst: entity work.ZeroFiller
    port map(IR => IR_sig, ZeroFilled_8 => Constant_Out);

    IDC_inst: entity work.InstructionDecoderController
    port map(
        RESET => RESET, CLK => CLK,
        IR => IR_sig, V => V, C => C, N => N, Z => Z,
        PS => PS_sig, IL => IL_sig,
        DX => DX, AX => AX, BX => BX, FS => FS,
        MB => MB, MD => MD, RW => RW, MM => MM, MW => MW
    );
end MCU_Structural;
```

Listing 3: MPC strukturel arkitektur (uddrag)

Et vigtigt designvalg er at PC-outputtet mappes direkte til `Address_Out` i port-mapping uden et mellemliggende signal. Da `Address_Out` er en ren udgang og ikke læses internt, er dette den simpleste løsning. Program Counterens ubrugte `Carry0`-output mappes til `open`.

6.3 Test

MPC topmodulet er verificeret med en integrations-testbench, der tester det samlede samspil mellem alle submoduler. I modsætning til IDC-testbenchen, hvor IR sættes direkte, drives her kun topmodulets eksterne signaler (`Instruction_In`, `Address_In`, statusflag). Instruktionen skal gennemløbe IR-loadning i INF-tilstanden før den kan eksekveres, hvilket verificerer hele den interne signalkæde.

6.3.1 Testmetode

Testbenchen følger samme gruppebaserede tilgang som IDC-testbenchen med en `TEST_GROUP` konstant (1–4). For hver instruktion er forløbet:

1. `Instruction_In` sættes til den ønskede instruktion (via `make_ir` hjælpefunktionen)
2. Første rising edge: INF → EX0, IR loader instruktionen
3. Efter 1 ns delta: EX0 kontrolsignaler verificeres med assert-statements
4. Anden rising edge: EX0 → INF, PC opdateres
5. `Address_Out` verificeres mod den forventede PC-værdi

En variabel `expected_pc` holder styr på den forventede PC-værdi og opdateres automatisk efter hver instruktion (inkrement, branch eller jump). Dette gør det muligt at verificere at PC-logikken fungerer korrekt i samspil med IDC'ens PS-signal.

6.3.2 Testgrupper

- **Gruppe 1 – Register-register:** Alle ti register-register operationer (MOVA, INC, ADD, SUB, DEC, OR, AND, XOR, NOT, MOVB). For hver instruktion verificeres FS, RW, DX, AX, MB samt at PC inkrementeres korrekt. Desuden verificeres `Constant_Out` fra Zero Filleren.
- **Gruppe 2 – Memory & Immediate:** LD, ST, ADI og LDI. Verificerer MD, MW, MM, MB og FS. For ADI og LDI kontrolleres at `Constant_Out` matcher det forventede zero-filled konstantfelt.
- **Gruppe 3 – Branch & Jump:** BRZ (taget og ikke-taget), BRN (taget og ikke-taget) og JMP. Her verificeres at `Address_Out` faktisk ændres korrekt: branch adderer Sign Extender-offsettet til PC, jump loader `Address_In`, og ikke-taget branch inkrementerer. Efter et jump verificeres at PC fortsat inkrementerer normalt.
- **Gruppe 4 – Multi-cycle:** LRI (3-cyklus), SRM med Z=1 (tidlig afslutning), SRM med Z=0 (fuld shift-løkke med EX2↔EX3) og SLM. Verificerer at PC holdes under hele flercyklus-forløbet og først inkrementeres ved afslutning. Kontrolsignaler (DX, AX, FS, MB, RW) verificeres i hver tilstand.

Simuleringsresultaterne for alle fire testgrupper er vist i sektion 7.2.

7 Syntese og Simulering

7.1 IDC Simulering

IDC testbenchen er opdelt i fire grupper (se testmetode i sektion 5.6). Nedenfor vises bølgeformerne fra Vivado XSim for hver gruppe.

7.1.1 Gruppe 1: Register-Register Instruktioner

Tester alle ti register-register operationer (MOVA, INC, ADD, SUB, DEC, OR, AND, XOR, NOT, MOVB). For hver instruktion verificeres at FS matcher den forventede funktionskode, at RW=1 (register write), og at PS=01 (PC increment).

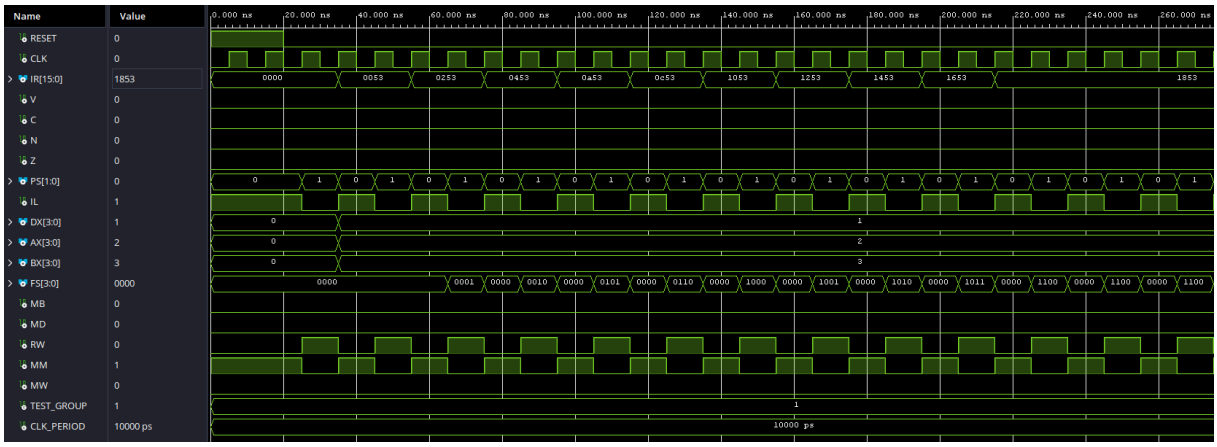


Figure 20: Simulering af Gruppe 1: Register-register operationer. IR skifter opcode for hver instruktion, og FS følger med den korrekte funktionskode.

7.1.2 Gruppe 2: Memory & Immediate

Tester LD, ST, ADI og LDI. Verificerer korrekt MD (memory data select), MW (memory write), MB (MUX B for immediate operander) og FS.

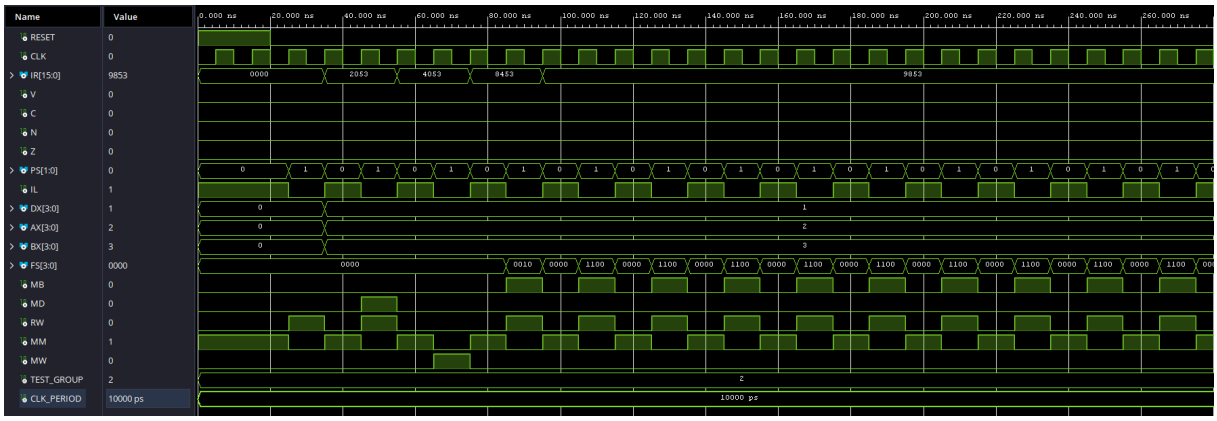


Figure 21: Simulering af Gruppe 2: Memory og immediate instruktioner.

7.1.3 Gruppe 3: Branch & Jump

Tester BRZ (branch on zero), BRN (branch on negative) og JMP. For branch-instruktionerne testes både taget og ikke-taget tilfælde ved at variere Z- og N-flag.

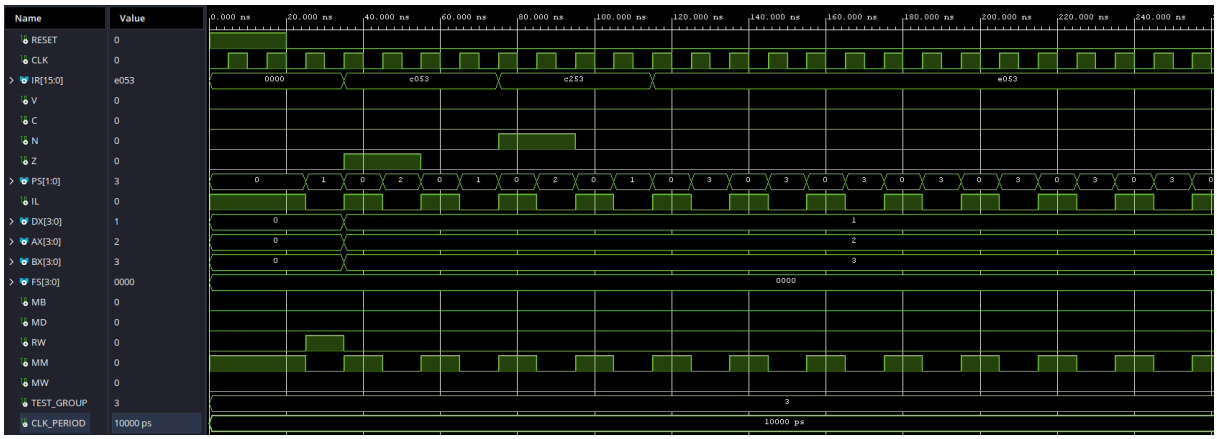


Figure 22: Simulering af Gruppe 3: Branch og jump instruktioner. PS skifter mellem 10 (branch taget) og 01 (ikke taget).

7.1.4 Gruppe 4: Multi-cycle (LRI, SRM, SLM)

Tester flercyklus-instruktionerne. LRI verificerer 3-cyklus forløbet (EX0: load til R8, EX1: load fra R8-adresse). SRM/SLM tester shift-løkken inklusive tidlig afslutning ved Z=1 og korrekt gennemløb af EX2↔EX3 løkken.

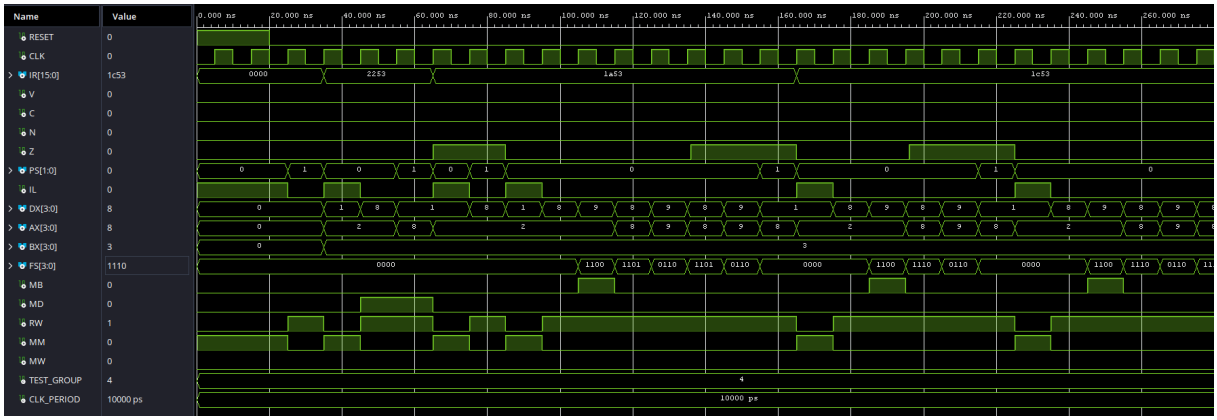


Figure 23: Simulering af Gruppe 4: Multi-cycle instruktioner. Bemærk de ekstra tilstande EX1–EX4 for SRM/SLM.

7.1.5 Verifikationsresultat

Alle fire testgrupper blev kørt i Vivado XSim. Ingen `Error`-beskeder blev rapporteret, hvilket bekræfter at samtlige 24 assert-statements bestod.

Gruppe	Beskrivelse	Antal tests	Resultat
1	Register-register	10	PASSED
2	Memory & Immediate	4	PASSED
3	Branch & Jump	5	PASSED
4	Multi-cycle (LRI, SRM, SLM)	5	PASSED
	Total	**24**	**PASSED**

Table 6: Testresultater for IDC testbench. Alle grupper bestod uden fejl.

7.2 MPC Top-Level Simulering

MPC topmodulets testbench verificerer det samlede samspil mellem alle submoduler. Testbenchen er opdelt i fire grupper (se testmetode i sektion 6.3).

7.2.1 Gruppe 1: Register-Register Instruktioner

Tester alle ti register-register operationer gennem hele signalkæden: `Instruction_In` → IR → IDC → kontrolsignaler. For hver instruktion verificeres at FS, RW, DX og AX er korrekte, at `Constant_Out` matcher Zero Fillerens output, og at `Address_Out` (PC) inkrementeres efter hver instruktion.

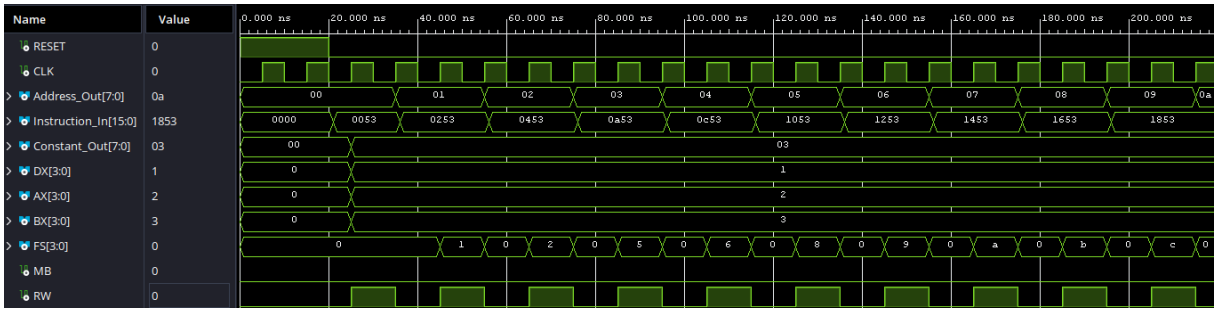


Figure 24: Simulering af MPC Gruppe 1: Register-register instruktioner. Address_Out inkrementeres fra 00 til 0A gennem ti instruktioner.

7.2.2 Gruppe 2: Memory & Immediate

Tester LD, ST, ADI og LDI gennem topmodulet. Verificerer at MB=1 for immediate-instruktioner, at `Constant_Out` indeholder det korrekte zero-filled konstantfelt, og at PC inkrementeres.

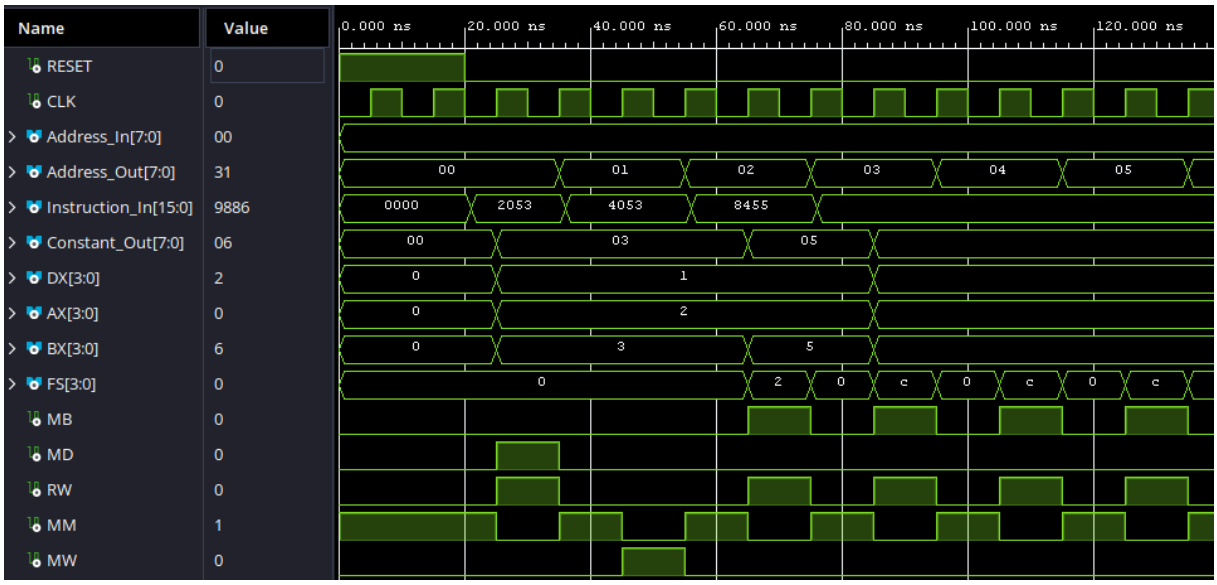


Figure 25: Simulering af MPC Gruppe 2: Memory og immediate instruktioner.

7.2.3 Gruppe 3: Branch & Jump

Tester branch- og jump-instruktioner med fokus på `Address_Out`-verifikation. BRZ med Z=1 udfører branch med Sign Extender-offset (PC ← PC + 3 = 03). BRZ med Z=0 inkrementerer normalt. BRN med N=1 brancher (PC ← PC + 10 = 0E). JMP loader `Address_In` = 42 direkte i PC. Efterfølgende MOVA bekræfter at PC fortsat inkrementerer normalt efter jump (43).

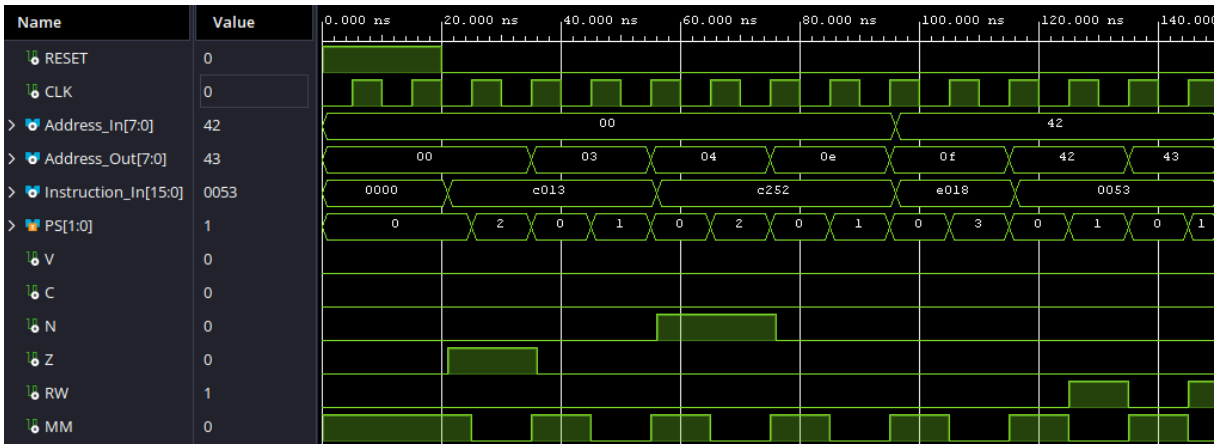


Figure 26: Simulering af MPC Gruppe 3: Branch og jump. Address_Out følger sekvensen 00 → 03 → 04 → 0E → 0F → 42 → 43.

7.2.4 Gruppe 4: Multi-cycle

Tester flercyklus-instruktionerne LRI, SRM og SLM. Verificerer at `Address_Out` holdes konstant under hele flercyklus-forløbet og først inkrementeres ved afslutning. SRM med Z=0 gennemløber den fulde shift-løkke (EX0 → EX1 → EX2 ↔ EX3 → EX4 → INF) mens PC forbliver uændret.

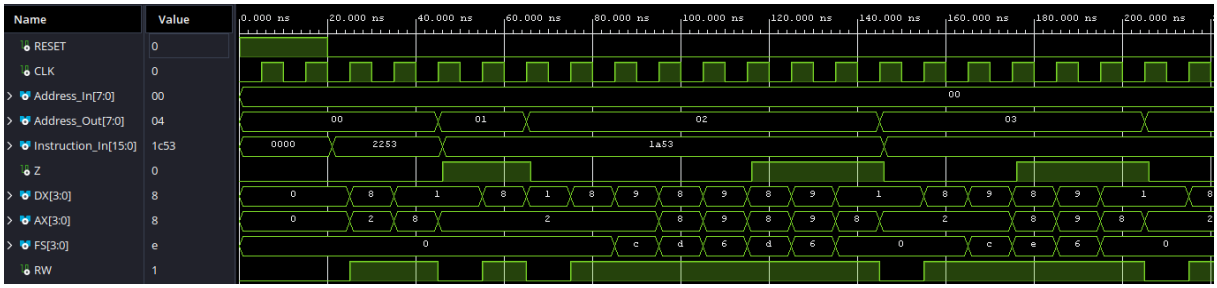


Figure 27: Simulering af MPC Gruppe 4: Multi-cycle instruktioner. Address_Out holder sin værdi under flercyklus-forløb.

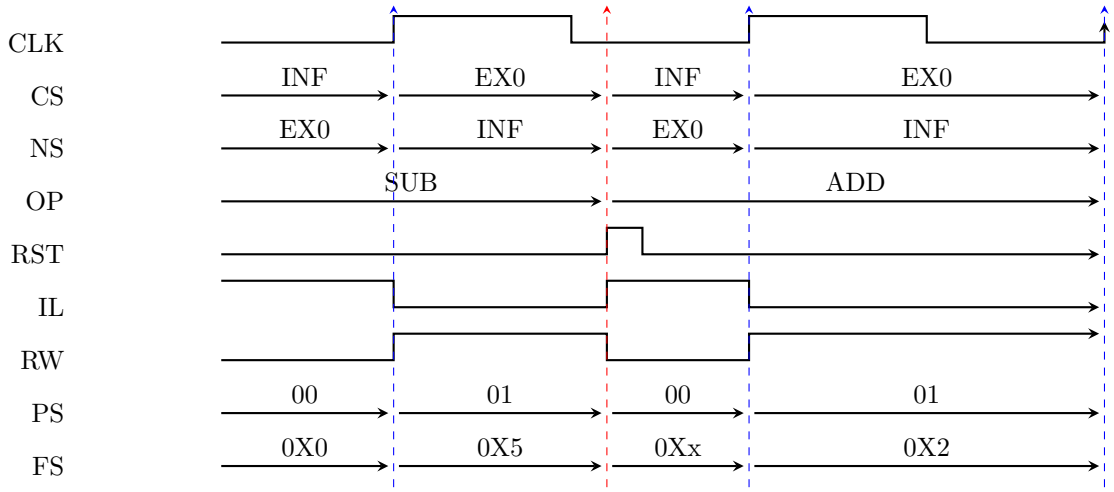
7.2.5 Verifikationsresultat

Alle fire testgrupper blev kørt i Vivado XSim. Ingen `Error`-beskeder blev rapporteret.

Gruppe	Beskrivelse	Antal tests	Resultat
1	Register-register	10	PASSED
2	Memory & Immediate	4	PASSED
3	Branch & Jump	5	PASSED
4	Multi-cycle (LRI, SRM, SLM)	5	PASSED
	Total	**24**	**PASSED**

Table 7: Testresultater for MPC top-level testbench. Alle grupper bestod uden fejl.

7.3 Verificering af Timing Diagram



For at verificere det viste timingdiagram for IDC'en er der udarbejdet en dedikeret VHDL testbench, som tester `SUB`, `reset` og `ADD`.

Simuleringen bekræfter, at reset korrekt tvinger FSM'en tilbage i INF-tilstanden (kan ses på `IL`), og at efterfølgende instruktioner udføres med korrekt timing for IL, RW, PS og FS i overensstemmelse med timingdiagrammet. Simuleringen kan ikke direkte følge `current_state` og `next_state`, da disse er enum-variabler. Tilstandene kan derfor ikke verificeres direkte, men de observerede kontrolsignaler understøtter, at IDC'en befinder sig i den forventede tilstand.

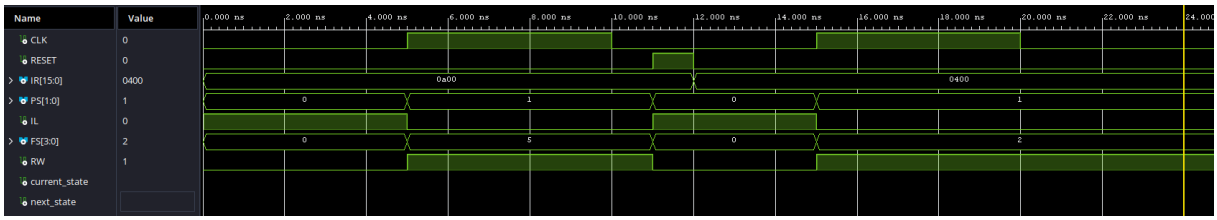


Figure 28: Billedet viser simulerings resultaterne af Timingdigram_TB

8 Konklusion

I dette projekt er der designet, implementeret og testet en komplet mikroprogram-controller i overensstemmelse med specifikationerne for Project Work B i kurset 62711 – Design of Digital Systems. Controlleren består af en Program Counter, Instruction Register, Sign Extender, Zero Filler samt en Instruction Decoder, som tilsammen udgør styringsdelen af et simpelt mikroprogrammeret system.
Alle moduler er designet strukturelt og implementeret i VHDL med udgangspunkt i projektets top-down-arkitektur, men udviklet ved en bottom-up tilgang med selvstændig test af hvert delmodul. Funktionaliteten af Program Counterens fire driftsformer (hold, increment, branch og jump) er verificeret både via timing-diagrammer og simuleringer. Instruction Registeret, Sign-Extenderen og Zero-Filleren er ligeledes testet og opfylder de specificerede krav.
Instruction Decoder er implementeret som en Mealy-type finit state machine og understøtter både simple to-cyklus instruktioner samt mere komplekse fler-cyklus instruktioner som LRI, SRM og SLM. State-diagrammer og kontrolord er udarbejdet og implementeret direkte i overensstemmelse med projektbeskrivelsen. Funktionaliteten er verificeret ved hjælp af en assert-baseret testbench, som automatisk kontrollerer samtlige relevante kontrolsignaler.
Samlet set viser simuleringerne, at alle moduler samt top-niveauet fungerer korrekt, og at alle testcases gennemføres uden fejl. Projektet har givet en øget forståelse for mikroprogrammerede controllere, samspillet mellem kontrol og datavej samt betydningen af systematisk test og verifikation i digitalt design.

Modulansvar

Modul (VHDL + TB)	**Ansvarlig**	**Subgruppe**
ProgramCounter	Jonas	2
CounterLogic, flip_flop, full_adder	Andreas & Jonas	2
InstructionRegister	Andreas	2
SignExtender, ZeroFiller	Jonas	2
InstructionDecoderController	Mads & Sigurd	1
MicroprogramController (top-level)	Mads & Jonas	Fælles
Rapport	Alle	Fælles

Table 8: Fordeling af modulansvar

Tidsforbrug

Navn	**Studienr.**	**Timer**
Andreas Skånning	s241123	6
Jonas Beck Jensen	s240324	6
Mads Rudolph	s246132	6
Sigurd Hestbech Christiansen	s245534	6
Total		**24**

Table 9: Estimeret tidsforbrug per gruppemedlem